Compte rendu

Ouarezki Oussama Abde Rahim G3

Introduction

In this report, we will use a **Support Vector Machine (SVM)** to predict whether a user will purchase a product based on their **age** and **estimated salary**. The dataset **Social_Network_Ads.csv** includes user details such as age, salary, and purchasing behavior (target variable). We will preprocess the data, build an SVM model, visualize the results, and evaluate the model's performance using accuracy and a confusion matrix.

The math behind SVM

A **Support Vector Machine (SVM)** is a classification algorithm that aims to find a **hyperplane** that best separates the data into two classes. The objective is to maximize the **margin**, which is the distance between the hyperplane and the closest data points, known as the **support vectors**.

Mathematically, the hyperplane is represented by:

$$\mathbf{w} \cdot \mathbf{x} + b = 0$$

The goal is to **minimize** (it's the inverse of the distance between tha margin):

$$\frac{1}{2}||\mathbf{w}||^2$$

subject to the constraint that each data point is correctly classified, i.e.,

$$y_i(\mathbf{w} \cdot \mathbf{x}_i + b) \geq 1$$

Finally, the model uses the decision function to predict new data points:

$$f(\mathbf{x}) = \mathbf{w} \cdot \mathbf{x} + b$$

If $f(\mathbf{x})$ is positive, the point is classified as one class (1) if negative, it is classified as the other class (0).

> Question 1
>
> Lisez l'ensemble de données Social_Network_Ads.csv, puis séparez les données en variables explicatives X (Age et EstimatedSalary) et en variable cible y (Purchased).

We import these three libraries as we will need them in this analysis, And we import our **csv file** in Variable called **data**.

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
data=pd.read_csv('Social_Network_Ads.csv')

data.head()
```

	User ID	Gender	Age	EstimatedSalary	Purchased
0	15624510	Male	19.0	19000.0	0
1	15810944	Male	35.0	20000.0	0
2	15668575	Female	26.0	43000.0	0
3	15603246	Female	27.0	57000.0	0
4	15804002	Male	19.0	76000.0	0

We splited our data according to the question.

```
X = data[['Age', 'EstimatedSalary']]
y = data['Purchased']
```

> Question 2
>
> Divisez l'ensemble de données en **Training set** et Test set en utilisant la fonction train_test_split, en prenant 20 % des données pour l'ensemble de test. Ensuite, appliquez le feature scaling (mise à l'échelle des caractéristiques) sur X_train et X_test en utilisant la méthode StandardScaler de Scikit-Learn.

In this step, we divide our dataset into two parts: **training set** and **testing set**, using the **train test split** function from sklearn. The **training set** is used to train the model, while the **testing set** is reserved to evaluate its performance.

After splitting the data, we scale the features using **StandardScaler**. This scaler standardizes the data by subtracting the mean of each column (feature) and then dividing by the standard deviation of that column. By subtracting the mean, we **center the data around 0**. Dividing by the standard deviation not only s**cales the data but also removes the units from the features**. This transformation ensures that all features have a mean of 0 and a standard deviation of 1, which helps to improve the performance and stability of many machine learning algorithms. This standardization is applied to each feature individually.

The formula :

$$z_{ij} = \frac{x_{ij} - \mu_j}{\sigma_j}$$

Where:
- z_{ij} is the standardized value for the j-th feature of the i-th data point.
- x_{ij} is the original value of the j-th feature for the$i $-th data point.
- μ_j is the mean of the j-th feature.
- σ_j is the standard deviation of the j-th feature.

```
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.20, random_state=42)

scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)
```

Here's an improved version of your explanation:

You might wonder why, for the training set, we use **X_train = scaler.fit_transform(X_train)**, but for the testing set, we only use **scaler.transform(X_test)**. The reason for this is that the testing set represents unseen data. We don't want to fit the scaler to the testing set because that would involve using information from the test data (like its mean and standard deviation), which could lead to **data leakage**.

Instead, we **fit the scaler to the training set** to compute the mean and standard deviation, and then use these statistics to **transform** both the training and testing sets. This ensures

that the transformation of the testing set is based on the training data only, preserving the integrity of the evaluation process.

> Question 3
>
> En utilisant l'ensemble du train, réalisez un scatter plot pour visualiser la relation entre Age, EstimatedSalary, et Purchased:
> - Les utilisateurs qui n'ont pas acheté (classe 0) en rouge.
> - Les utilisateurs qui ont acheté (classe 1) en bleu.
>
> Que pouvez-vous dire sur la séparation des classes en fonction de l'âge et du salaire estimé ?

In this part we want to plot *Age* in X axis and *Estimated salary* in the Y axis and color them when they are blue when they bought the product and red if they don't and to do that we use this expresssion **y_train == 1** if they boughet the producte since it is a binary and **y_train == 0** if they didn't and those expression will return a boolean vector adn since the **X_train** and the **Y_train** are aligned, we can simply use **X_train[y_train == 0]** to select only the individuals who didn't buy the product and vice versa.

and you will see by this example:

```
A=np.array([[13,1],[7,0]])
A=pd.DataFrame(A)
A
```

	0	1
0	13	1
1	7	0

if i want to this array to return the value **13** using the methode that i explained earlier we simply give it the condition **B==1**

```
B=A.iloc[:,1] # first column
C=A.iloc[:,0] # second column
C[B==1]
```

```
0    13
Name: 0, dtype: int32
```

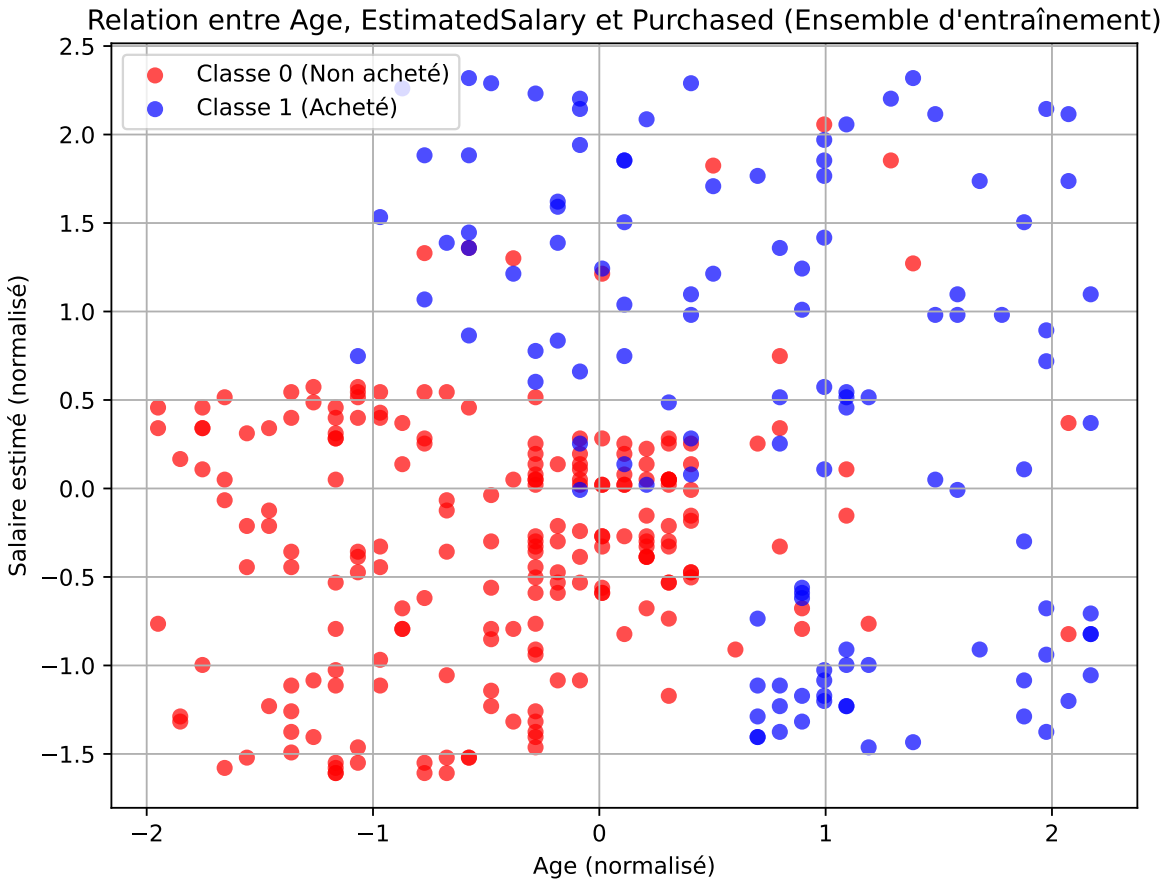
Now we apply it here and we get the **scatter plot**

```
plt.figure(figsize=(8, 6))

plt.scatter(X_train[y_train == 0][:, 0], X_train[y_train == 0][:, 1], color='red', label='Cla

plt.scatter(X_train[y_train == 1][:, 0], X_train[y_train == 1][:, 1], color='blue', label='Cl

plt.title("Relation entre Age, EstimatedSalary et Purchased (Ensemble d'entraînement)")
plt.xlabel("Age (normalisé)")
plt.ylabel("Salaire estimé (normalisé)")
plt.legend()
plt.grid(True)
plt.show()
```



Question 4

Construire un modèle de classification SVM linéaire :
- Importez la classe SVC depuis la bibliothèque sklearn.svm. - Créez un modèle SVM en utilisant le noyau linéaire (kernel='linear'), puis entraînez le modèle sur l'ensemble d'entraînement avec la méthode .fit().

Here, we imported the *Support Vector Classification* (SVC) from **sklearn** and set the kernel to 'linear', as specified in the question, And then we simply fitted the model.

```
from sklearn.svm import SVC


svm_linear = SVC(kernel='linear', random_state=13)


svm_linear.fit(X_train, y_train)
```

```
SVC(kernel='linear', random_state=13)
```

Question 5 (6)

Utilisez le modèle que vous avez créé pour faire des prédictions sur l'ensemble de test. Affichez ensuite les classes prédites pour les comparer aux classes réelles

For the predection we simply use the methode **predict()** and we stored it in variable called **y_pred**.

And you can see that we add the methode **.values** because we want to transform our **y_test** in to numpy array so it can match **y_pred**

```
y_pred = svm_linear.predict(X_test)

type(y_test.values)==type(y_pred)
```

```
True
```

And here we simply summed the values that are the same which the model got right **TP** and we we summed the values where the model got wrong which are **FP**.

```
print("Real classes :")
print(y_test.values)

print("\n Predicted classes :")
print(y_pred)

print('\n Number of classes that the model got right:',(y_test.values ==y_pred).sum())

print('\n The number of classes that the model got wrong',(y_test.values !=y_pred).sum())
```

```
Real classes :
[0 1 0 1 0 0 1 0 0 0 0 1 0 0 0 0 1 0 0 1 0 0 1 1 0 1 0 0 1 0 1 0 1 0 1 0 0
 0 0 0 1 0 0 1 0 1 0 0 1 0 0 1 0 0 0 0 1 0 0 0 0 0 1 0 0 1 1 1 0 0 1 0 0 0
 1 0 1 1 0 1]

 Predicted classes :
[0 1 0 1 0 0 1 0 0 0 0 1 0 0 0 0 1 1 0 1 0 0 0 1 0 0 1 0 1 0 0 0 1 0 1 0 0
 0 0 0 0 0 0 0 0 1 0 0 0 0 0 1 0 0 0 0 1 0 0 0 0 0 1 0 0 0 1 1 0 0 1 0 0 0
 0 0 1 1 0 0]

 Number of classes that the model got right: 69

 The number of classes that the model got wrong 11
```

> Question 6
>
> Tracez un graphique 2d avec les points du dataset X_train et la droite séparatrice qui sépare les deux classes (classe 0 et classe 1).

As of the graph of the scatter plot we explained earlier in Question 3.

the SVM line is given by the equation:

$$b + w_1.X_1 + w_2.X_2 = 0$$

We rearrange the equation to solve for x_2:

$$x_2 = -\frac{w_0 \cdot x_1 + b}{w_1}$$

Similarly, for a range of x_2 values, we compute the corresponding x_1 values using:

$$x_1 = \frac{-w_1 \cdot x_2 - b}{w_0}$$

Both approaches allow us to draw the line of the SVM.

So we applied the first approche we we got both of the coeffcient using **svm_linear.coef_** and the intercept using **svm_linear.intercept_** and we set the values of x_1 in range of $[min(x_{1,i}), max(x_{1,i})]$ from the first column of X which is X_1, and then we simply plotted everything.

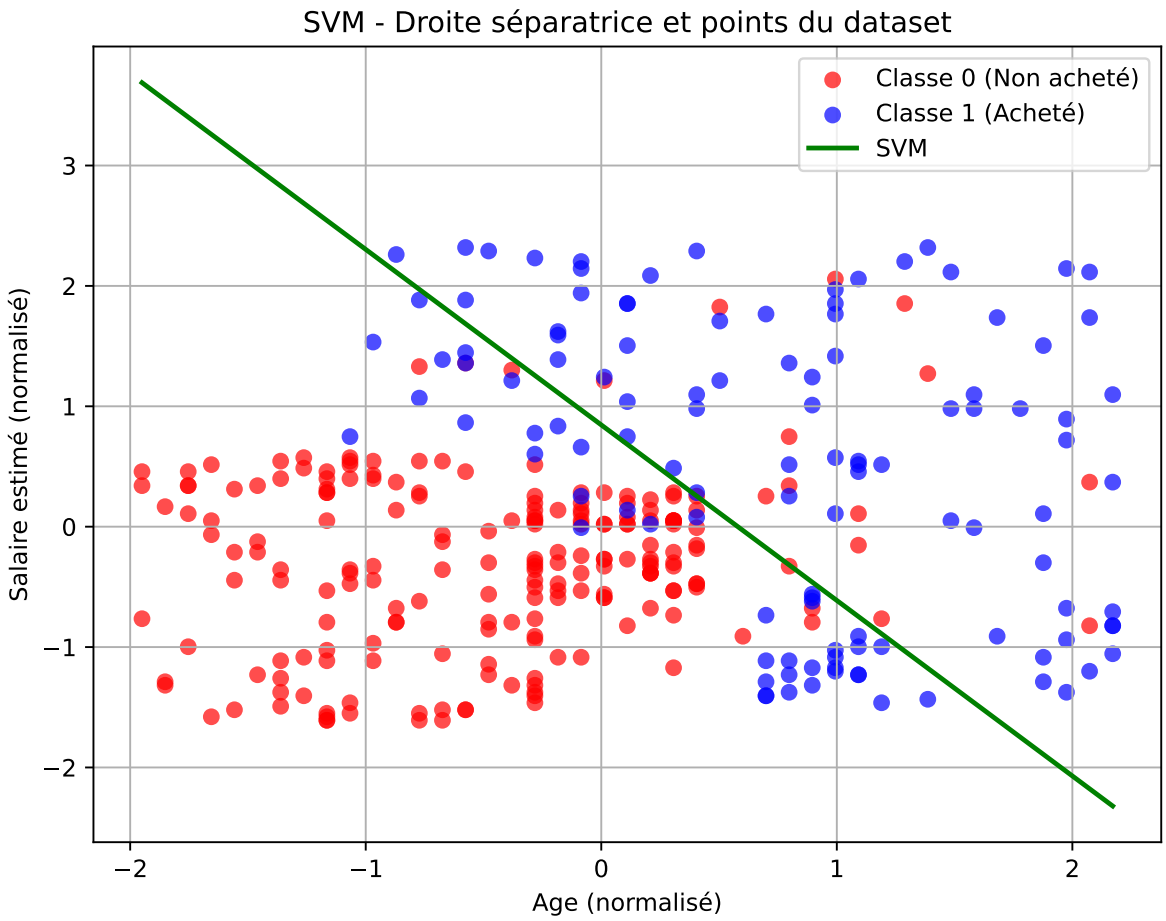
```
plt.figure(figsize=(8, 6))
plt.scatter(X_train[y_train == 0][:, 0], X_train[y_train == 0][:, 1], color='red', label='Cla
plt.scatter(X_train[y_train == 1][:, 0], X_train[y_train == 1][:, 1], color='blue', label='Cl


w = svm_linear.coef_
b = svm_linear.intercept_


x1_range = np.linspace(X_train[:, 0].min(), X_train[:, 0].max(), 100)
x2_range = -(w[0][0] * x1_range + b) / w[0][1]


plt.plot(x1_range, x2_range, color='green', label='SVM', linewidth=2)


plt.title("SVM - Droite séparatrice et points du dataset")
plt.xlabel("Age (normalisé)")
plt.ylabel("Salaire estimé (normalisé)")
plt.legend()
plt.grid(True)
plt.show()
```



Question 7

Utilisez confusion_matrix de sklearn.metrics pour obtenir la matrice de confusion du modèle, puis mesurez l'accuracy sur l'ensemble de test.

To calculte the **Confusion Matrix** and the **accuracy_score** we need to import them from the library **sklearn**

```
from sklearn.metrics import confusion_matrix, accuracy_score,f1_score

cm = confusion_matrix(y_test, y_pred)
print("Matrice de confusion :")
print(cm)


accuracy = accuracy_score(y_test, y_pred)
```

```
print("\nAccuracy :")
print(accuracy)
```

```
Matrice de confusion :
[[50  2]
 [ 9 19]]

Accuracy :
0.8625
```

And from these resulats we can see that the **TP** of the first variable are more the **TP** of the second variable which means that the accuracy of the first variable is effecting and like supressing the overall accuracy which in this case the **accuracy score** isn't not reliable in this case we have to use the **F1-Score** it give more accurate resulats.

```
f1=f1_score(y_test,y_pred)
print("The F1-score is:",f1)
```

```
The F1-score is: 0.7755102040816326
```

So in this case 0.77 is more accurate than 0.86 and this is more logical of we look at the graph and the line of svm.

Conclusion

Conclusion

In this report, we built a **Support Vector Machine (SVM)** model to predict whether a user would purchase a product based on their **age** and **estimated salary**. By following a structured approach, we preprocessed the data, divided it into training and testing sets, and standardized the features using **StandardScaler**. We then visualized the relationship between the features and the target variable, which helped us understand the separation of the classes.

The model was trained using a linear kernel, and we made predictions on the test set. The results were evaluated using **accuracy** and the **confusion matrix**, which provided insight into how well the model performed. Additionally, we calculated the **F1-score**, which gave a more balanced measure of performance, especially considering the potential class imbalance.

Finally, the performance of the model, based on the given dataset, showed that the SVM model is effective for classification tasks like this one, but it is important to evaluate multiple metrics (such as F1-score) to ensure reliable results.